Crash-safety discipline in `ConnectRedirectClassify`

A bug in most code crashes a process. A bug in this function — kernel-mode, on the network data path, invoked on every outbound `connect()` that matches a redirect filter — bugchecks the whole machine. This document is an honest accounting of what has actually been done to keep that from happening, and equally honest about what has **not** been done, so a reviewer (or a future version of this document) isn't misled into treating "compiled and reasoned about" as "proven safe."

Status as of this writing: **compiled against WDK 10.0.26100, never loaded, never exercised.** Everything below is design-time and review-time discipline, not verified runtime behavior.

What's actually been done

The safe action is set first, unconditionally, before any logic runs. `classifyOut->actionType = FWP_ACTION_PERMIT` is the very first statement in the function. Every one of its many early-return paths — shutting down, no `layerData`, no redirect handle, rights already claimed, no target configured, loop-guard hit, handle-acquisition failure, writable-pointer failure — is simply `return;` with nothing else to undo. A future bug that adds a wrong early return, or removes a check, still leaves the connection permitted unchanged. There is no path, reachable by any combination of these checks, that leaves `classifyOut` in an unset or ambiguous state.

The function cannot block, structurally, not just by convention. Its entire action vocabulary is binary: modify the connect request and permit, or permit unchanged. `FWP_ACTION_BLOCK` (or any denying action) never appears in this function at all. The worst a logic bug here can do is misroute a connection or fail to redirect it — never sever it. This is a narrower failure mode than the existing SNI/STREAM callout, which has to be able to block by design and carries that risk deliberately.

No heap allocation anywhere in this path. The SNI/STREAM classify function calls `ExAllocatePool2` for pended-flow records and held stream segments, and has to handle OOM on each one. `ConnectRedirectClassify` allocates nothing — no pool tag, no allocation failure to handle, no use-after-free surface from a buffer freed on one path and read on another. Fewer resources touched means fewer ways to get resource lifetime wrong.

Fail-open is checked explicitly, early, and before anything is touched: `g_ShuttingDown`, `layerData == NULL`, `g_RedirectHandle == NULL`, `FWPS_RIGHT_ACTION_WRITE` not held, `{pid, port}` unpacking to zero, the redirect-loop-guard state, `FwpsAcquireClassifyHandle0` failure, and `FwpsAcquireWritableLayerDataPointer0` failure — every one of these is a guard clause that returns before any mutation. None of them are "handle the error and continue"; they are all "stop here."

The struct layout and API semantics were checked against the real installed WDK headers, not transcribed from memory or the design doc's prose — and two things the prose got wrong were caught this way: `localRedirectTargetPID` is a `DWORD`, not a `HANDLE`; and there are two distinct "redirected by self" enum members (`FWPS_CONNECTION_REDIRECTED_BY_SELF` and `FWPS_CONNECTION_PREVIOUSLY_REDIRECTED_BY_SELF`), both now handled by the loop guard. A wrong field type or a missed enum member in this category of API is a type-confusion or silent-logic bug, not something that

fails loudly at compile time. `in6addr_loopback` was deliberately avoided (it pulls in a separate link library — the same reason Windows' own `IN6_IS_ADDR_LOOPBACK` avoids it) in favor of constructing `::1` by hand, removing a plausible link-time failure mode entirely.

No new global mutable state beyond one read-mostly handle. The only process-wide state this feature owns is `g_RedirectHandle` — created once, at `PASSIVE_LEVEL`, during `RegisterCallouts`, and destroyed once at unload. There is no counter, table, or lock introduced by this feature, and — since the per-agent target moved from two global variables to each matched filter's own `context` field — there is now **no write path into this feature's runtime state from inside a classify invocation at all**. Two CPUs running this classify function concurrently have nothing of this feature's own to race on.

Build-level discipline matches the existing driver's bar. `/kernel /GS /W4 /WX` — kernel-mode compilation, stack-protection cookies, and every warning promoted to an error, so an uninitialized read or an implicit narrowing conversion fails the build instead of shipping quietly. This isn't new for this feature; it's the same `build.cmd` flags the whole file already builds under.

Separable from the already-further-along SNI/STREAM datapath. The two features share only the WFP provider/sublayer GUIDs (an organizational grouping) and the device/module they're compiled into. No runtime state, lock, or data structure is shared between them.

What has explicitly NOT been done

- **Never loaded.** Not on a test-signed VM, not under a debugger, not once. Everything above is static review and type-checking, not observed behavior.
- **No Driver Verifier pass.** The existing SNI driver's own validation checklist still lists this as open for its half of the file; the redirect half hasn't reached even that stage.
- **No fuzzing, no concurrency/load test.** Nothing has exercised two simultaneous redirected connections, a listener restart mid-flow, or a malformed/adversarial connect attempt against this code.
- **IRQL correctness is "follows the documented API contracts," not independently verified.** Every call used (`FwpsAcquireClassifyHandle0`, `FwpsAcquireWritableLayerDataPointer0`, `FwpsQueryConnectionRedirectState0`, etc.) is used within its documented `_IRQL_requires_max_` bound, but no tool has confirmed that at runtime.
- **Code signing is unrelated to all of the above and doesn't substitute for it.** EV/attestation/WHQL signing proves provenance, not correctness. A perfectly signed driver with a logic bug still bugchecks the machine.
- **Test-signing and loading are scoped to a disposable VM, deliberately never a primary machine** — precisely because none of the above has been verified yet, and the failure mode if something's wrong is "the machine goes down," not "the process restarts."

The one thing this document can't claim

Nothing above defends against memory corruption that originates **outside** this function's own code — a wild write from unrelated kernel code elsewhere in this module, or another driver entirely, could still corrupt `g_RedirectHandle` or any other global. That's a general property of running in kernel mode at all, not something a single classify function can guarantee for itself. The measures above are about this function's own logic being unable to produce a bad outcome on its own, not about the machine being safe from every other

actor in Ring 0.

Before this is loaded anywhere, even the disposable VM

1. Attach WinDbg (kernel) before the first `sc start`, same as the README already requires for the SNI half.
2. Run under Driver Verifier (WFP + pool tracking) for the whole pass, not just the SNI callouts.
3. Validate the mechanism against a single throwaway `127.0.0.1` echo listener — one connection, manually driven — before anything resembling real agent traffic.
4. Confirm the unload path (filter delete, callout unregister, redirect handle destroy) leaves no dangling WFP objects by re-running the whole sequence (load -> exercise -> unload -> load again) rather than loading once and calling it done.